

此次我实现了基于字的二元模型和三元模型的拼音输入法，具体如下：

1. 实验环境

1.1 硬件环境

- CPU: Intel Core i7-13700HX
- 内存: 16GB DDR4
- 操作系统: Windows 11 64位

1.2 软件环境

- 编程语言: Python 3.10
- 开发工具: Visual Studio Code
- 依赖库:
 - pypinyin: 用于将汉字转换为拼音。
 - tqdm: 用于显示进度条。
 - json: 用于处理 JSON 格式的数据。
 - argparse: 用于解析命令行参数。
 - math: 用于对数运算。

2. 数据预处理

1. 使用的语料库： 新浪新闻语料库 (`sina_news_gbk`)， 包含 2016 年 4 月至 11 月的新闻数据。
2. 预处理后文件存储位置： 预处理后的文件均保存在 `sets/` 目录下
3. 生成拼音-汉字表：
 - 遍历拼音汉字表中的每一行， 过滤掉不在一二级汉字表中的汉字。
 - 结果保存在 `sets/word_table.json` 中， 格式为 `{<拼音>: [<汉字>, <汉字>, ...]}`。

4. 预处理新浪语料库：

- 对每条新闻，提取其中的 `title` 和 `html` 部分，将每个部分作为一个学习语料。
- 使用 `pypinyin` 库对预料进行拼音标注。即采用库中的 `lazy_pinyin()` 方法将汉字转换为拼音，遇到非汉字字符时用 `'*'` 占位。
- 结果保存在 `sets/processed_data/month_%02d.json` 中，格式为 `{<句子>: <拼音序列>}`。

5. 统计一元、二元、三元词频：

- 读取与处理后的新浪语料库。遍历每个月的数据，统计每个拼音对应的汉字及其出现次数，生成一元词频表。
- 同时统计相邻拼音对对应的汉字对及其出现次数，生成二元、三元词频表。
- 将结果保存到 `sets/1_word.json` , `sets/2_word.json` 和 `sets/3_word.json` 文件中。

3. 实验模型

3.1 二元模型公式推导

对给定的一个拼音序列 $\{p_1, p_2, \dots, p_n\}$ ，其中 p_i 为其中第 i 个汉字的拼音，我们希望得到的是在给定这个拼音序列条件下，最高概率的汉字序列 $\{w_1, w_2, \dots, w_n\}$ （即句子），即：

$$W_0 = \arg \max_{w_1, w_2, \dots, w_n} P(w_1 w_2 \dots w_n | p_1 p_2 \dots p_n)$$

根据贝叶斯公式，我们有：

$$P(w_1 w_2 \dots w_n | p_1 p_2 \dots p_n) = \frac{P(p_1 p_2 \dots p_n | w_1 w_2 \dots w_n) \cdot P(w_1 w_2 \dots w_n)}{P(p_1 p_2 \dots p_n)}$$

由于 $\{p_1, \dots, p_n\}$ 序列是确定的，故分母 $P(p_1 p_2 \dots p_n)$ 为常数，而 $P(p_1 p_2 \dots p_n | w_1 w_2 \dots w_n)$ 可认为是 1（保证生成的汉字一定对应其读音），因此我们只须关注 $P(w_1 w_2 \dots w_n) = P(W)$
通过条件概率可化简为：

$$\begin{aligned}
W_0 &= \arg \max_{w_1, w_2, \dots, w_n} P(W) \\
&= \arg \max_{w_1, w_2, \dots, w_n} P(w_1 w_2 \dots w_n) \\
&= \arg \max_{w_1, w_2, \dots, w_n} P(w_1) P(w_2 | w_1) P(w_3 | w_1 w_2) \dots P(w_n | w_1 w_2 \dots w_{n-1}) \\
&\approx \arg \max_{w_1, w_2, \dots, w_n} P(w_1) P(w_2 | w_1) P(w_3 | w_2) \dots P(w_n | w_{n-1}) \\
&= \prod_{i=1}^n P(w_i | w_{i-1})
\end{aligned}$$

其中 w_i 表示第 i 个汉字的预测， $P(w_i | w_1 w_2 \dots w_{i-1})$ 表示前 $i - 1$ 个字分别确定为 $w_1, \dots, w_{i-1}$ 时，第 i 个字为 w_i 的概率， $P(w_i | w_{i-1})$ 表示前一个字为 w_{i-1} 时，后一个字为 w_i 的概率（在给定拼音条件下），注意：对首字的处理是默认 $P(w_1 | w_0) = P(w_1)$ 即第一个拼音对应 w_1 的概率。

用语料中的统计频率来估计该二元条件概率，即：

$$P(w_i | w_{i-1}) \approx \frac{\text{count}(w_{i-1} w_i)}{\text{count}(w_{i-1})} = P^*(w_i | w_{i-1})$$

其中 $\text{count}(w_{i-1} w_i)$, $\text{count}(w_{i-1})$ 分别表示语料中 $w_{i-1} w_i$ 和 w_i 出现的频次。为防止概率为0的情况，对该条件概率作平滑处理，得到：

$$P(w_i | w_{i-1}) \approx \alpha P^*(w_i | w_{i-1}) + (1 - \alpha) P^*(w_i), \alpha \approx 1$$

其中 $P(w_i)$ 的估计 $P^*(w_i)$ 为：

$$P^*(w_i) = \frac{\text{count}(w_i)}{\text{para}}$$

其中 para 表示所有字在语料库中出现的频率之和，即语料库的总字频数。

3.2 二元模型算法原理

设 $\text{dp}_{i,w}$ 表示前 i 个汉字，第 i 个为 w 的概率对数最大值， $\text{path}_{i,w}$ 则表示此时选择的第 $i - 1$ 个汉字。

初始化：

$$\text{dp}_{1,w} = \log P(w), w \in \text{pinyin}(p_1)$$

$$\text{path}_{1,w} = \emptyset$$

其中 $\text{Word}(p_1)$ 表示拼音 p_1 所对应的汉字集合。

状态转移方程：

$$\begin{cases} dp_{i+1,w} = \max_{w' \in \text{Word}(p_i)} (dp_{i,w'} + \log P(w|w')) \\ \text{last}_{i+1,w} = \arg \max_{w' \in \text{Word}(p_i)} (dp_{i,w'} + \log P(w|w')) \end{cases}$$

通过动态规划求解得到完整的 dp 和 path 结果，在从最后一个字回溯path数组得到最优汉字序列。

3.3 三元模型

考虑通过前两个汉字来分析下一个汉字，即：

$$\begin{aligned} W_0 &= \arg \max_{w_1, w_2, \dots, w_n} P(w_1)P(w_2|w_1)P(w_3|w_1w_2) \cdots P(w_n|w_1w_2 \cdots w_{n-1}) \\ &\approx \arg \max_{w_1, w_2, \dots, w_n} P(w_1)P(w_2|w_1)P(w_3|w_1w_2) \cdots P(w_n|w_{n-2}w_{n-1}) \\ &= \prod_{i=1}^n P(w_i|w_{i-2}w_{i-1}) \end{aligned}$$

平滑处理：

$$\begin{aligned} P(w_i|w_{i-2}w_{i-1}) &\approx \gamma P^*(w_i|w_{i-2}w_{i-1}) + (1 - \gamma)P(w_i|w_{i-1}) & \gamma &\approx 1 \\ &\approx \gamma P^*(w_i|w_{i-2}w_{i-1}) + (1 - \gamma)[\alpha P^*(w_i|w_{i-1}) + (1 - \alpha)P^*(w_i)] & \alpha &\approx 1 \end{aligned}$$

其中：

$$P^*(w_i|w_{i-2}w_{i-1}) = \begin{cases} np \frac{\text{count}(w_{i-2}w_{i-1}w_i)}{\text{count}(w_{i-2}w_{i-1})} & \text{count}(w_{i-2}w_{i-1}) > 0 \\ np \cdot 0 & \text{otherwise.} \end{cases}$$

Viterbi算法的求解类似于二元的情况。需要注意的是，在初始化时， $dp_{1,w} = P(w)$ ， $dp_{2,v} = \alpha P(w_2|w_1) + (1 - \alpha)P(w_1)$

3.4 实验结果及案例分析

使用模型测试输入文件 `data/input.txt`，输出为结果文件为 `data/output.txt`。

3.3.1 实验结果

1. 实验结果：

- 平滑参数 α 取值为0.99999
- 二元模型经调参后句准确率为38.92%，字准确率为83.49%
- 三元模型取参数 $\gamma = 0.999$ ，句准确率为46.9%，字准确率为87.14%
(调参对比见后文)

2. 实验用时:

1. 二元模型:

- 训练总用时31min, 其中预处理新浪语料库25min, 计算一元及二元词频6分钟
- 生成测定样例用时10s

2. 三元模型:

- 预处理新浪语料库同二元模型, 25分钟
- 计算三元词频6分钟
- 生成测定样例用时1min30s

3.3.2 案例分析

1. 正确案例

- zhong guo pin kun di qu shi xian wang luo fu wu quan fu gai
中国贫困地区实现网络服务全覆盖
- wo men dou you guang ming de wei lai
我们都有光明的未来

2. 错误案例 (以下均为二元模型的结果)

- qing hua ci zhen zheng cheng wei zhong guo ci qi de zhu liu
输出: 清华慈真正成为中国瓷器的主流
答案: 青花瓷真正成为中国瓷器的主流
- zai huo po qi fen zhong ba jian kang zhi shi dai gei da jia
输出: 在活泼起分钟把健康知识带给大家
答案: 在活泼气氛中把健康知识带给大家
- chun feng hua yu le wei yang
输出: 春风化娱乐喂养
答案: 春风化雨乐未央
- wo cong wei jian guo you ru ci hou yan wu chi zhi ren
输出: 我从未见过犹如此后延误嗤之人
答案: 我从未见过有如此厚颜无耻之人

3. 错误案例在三元模型中的结果:

- qing hua ci zhen zheng cheng wei zhong guo ci qi de zhu liu
输出: 青花瓷真正成为中国瓷器的主流
- zai huo po qi fen zhong ba jian kang zhi shi dai gei da jia
输出: 在活泼气氛中吧健康知识带给大家

- chun feng hua yu le wei yang
输出：春风化雨了未央
- wo cong wei jian guo you ru ci hou yan wu chi zhi ren
输出：我从未见过犹如此厚颜无耻之人

4. 分析：

可以发现，二元模型对一些短语(如正确案例首句中的“中国”“贫困”)组成的句子有不错的表现。但一旦出现一些较长的固定短语，如“青花瓷”“厚颜无耻”或一些古文，二元模型会产生比较离谱的错误。

- 一方面，这源于二元模型本身算法的局限性。在只考虑前一个字的情况下，一句话中的上下文信息较短，对一些长短语，例如成语和古文，不能有效识别。
- 另一方面，训练语料本身的偏向性也可能是输出不正确的原因。比如“春风化雨乐未央”的识别失误可能与预料中相应古文出现频次较少有关
- 相比于二元模型，三元模型在上下文上有了明显的进步。上述错误案例通过三元模型优化后没有错误或错误率降低到个别字，已经不影响语义的理解，对长短语的解析明显优化。

5. 可能的改进方向：

对于后者，一些更大更全面的数据集或语料库可能可以较好地解决；

对于前者，则可能需要加入词多元模型或者考虑使用分词的模型，使得模型可以更深入理解句子本身。

4. 思考题

4.1 语料编码

1. GBK 和 UTF-8 的区别

- gbk编码：是经典的中文编码方式。字节长度要比utf-8短。基本只支持中文环境，所以更适合中文本土化环境，这种情况下会比utf-8编码效率更高。缺点就是不支持其他语言。
- utf-8编码：unicode编码，字符长度在1~4字节不等。优点就是兼容性很好，支持所有国际语言，所以跨语言适合使用。缺点就是在utf-8中中文字符是占3个字节的，所以只处理中文时效率不如gbk

2. 其他编码方式：

- **ASCII**: 很经典，适用于英文文本，单字节，效率高。
- **GB2312**: 简体中文，1-2 字节，可以在大陆用。
- **Big5**: 繁体中文，1-2 字节，适合港澳地区本地用。
- **UTF-16**: 2-4 字节，适合内存中存储 Unicode 字符串。

在本次作业中，由于处理的是中文语料，所以如果统一采用gbk编码也是可以的，且效率会更高。除此之外，采用utf-8和utf-16也可以，而且由于在utf-16中中文字符只占2字节，所以会比utf-8效率更高
总之，gbk适合本地，uff-8适合国际化。

4.2 算法设计

1. 时间复杂度分析

Viterbi 算法的核心是动态规划，计算每个拼音对应的所有可能汉字的概率，并记录路径。

对于输入的句子长度为 n ，字库大小为 V ，拼音数量为常数（设为 P ），考虑二元模型算法的循环：

- 外层循环遍历每个拼音（ n 次）。
 - 内层循环遍历当前拼音的所有可能汉字（最多 $\bar{P}$ 个）。
 - 对于每个汉字，还需要遍历前一个拼音的所有可能汉字（最多 $\bar{P}$ 个）。
- 而对于三元模型，还需要再遍历前第二个拼音，也是最多 $\bar{P}$ 个

因此时间复杂度为： $\left(n (\bar{P})^2\right)$

对于三元，时间复杂度为： $\left(n (\bar{P})\right)$

2. 空间复杂度分析

动态规划表 dp 和路径表 $path$ ：

- dp_i 存储第 i 个拼音对应的所有可能汉字及其概率，最多需要存储 V 个汉字。
- 总共需要存储 n 个拼音
- 空间复杂度为： (n) ，对二元和三元模型相同

4.3 输入输出

1. 第一种输入：每输入一行后立刻输出结果
2. 第二中输入：一次性输入多行后，再输出结果

在此次的OJ中，需要实现多行输入和对应输出，两种代码应该都是正确的

4.4 调参和模型对比

三元模型的原理和案例分析见上文。

1. 对二元模型进行调参

平滑参数 α	句准确率(%)	字准确率(%)
0.8	36.92	83.3
0.9	38.12	83.49
0.99	38.34	83.51
0.999	38.74	83.47
0.99999	38.92	83.49
0.999999	38.92	83.49

发现平滑参数越接近1，模型性能越好，在 α 超过0.9999之后，模型性能几乎不变。这和理论分析的结果基本吻合

2. 对三元模型进行调参

平滑参数 γ	平滑参数 α	句准确率(%)	字准确率(%)
0.9	0.99999	45.7	87.37
0.95		46.3	87.02
0.99		46.7	87.43
0.999		46.9	87.65

在接近1的情况下，三元平滑参数 γ 的变化对模型性能的影响较小

4.5 其他

1. 可能的评价指标：

在测试过程中，我发现输出的一些句子几乎是正确的，只有极个别字出现错误。可以考虑

- 增设指标：字准确率>90%的句子。
- 使用 jieba 等第三方库对答案和输出进行分词匹配

2. 小结：

这是我第一次做一个较大的项目。这个过程中收获了很多，如：编码知识、项目架构技巧、python语法和调试技巧等。关于拼音输入法这个项目，由于时间原因没能实现比三元模型更好的优化，但关于优化我有以下想法：

- 更多元的模型，但由此会带来算法复杂度问题
- 使用分词的模型，让模型对上下文语义信息有更好的表现
- 更丰富的语料库，除新浪新闻外，可以考虑知乎，b站等平台的语料